

Inteligencia Artificial y Mini-Robots

Taller 4

Presentado por:

Cristian Camilo García Gamboa - crigarciaga@unal.edu.co

Mateo Becerra Porras - mabecerrap@unal.edu.co

Santiago Ospina Contreras - saospinaco@unal.edu.co

Profesor:

José J. Martínez P.

jmcursosingenieria111@gmail.com

Fecha de entrega:

24 de Septiembre de 2026



Universidad Nacional de Colombia - Sede Bogotá
Facultad de Ingeniería
2026 II

De los ejercicios 1 y 2, escoja 1 para realizar; y hacer 3 o 4.

2. Suponga que tiene un robot que le entrega galletas al grupo de ingenieros de diseño de robots. Programe por PG el recorrido del robot, teniendo en cuenta que cada vez que un ingeniero recibe una galleta gana puntos. Los ingenieros están distribuidos en una sala cuadrada. Defina, conjunto de terminales, conjunto de funciones y función de aptitud.

El código a continuación desarrolla una solución de Programación Genética (PG) para el problema de un robot que debe repartir galletas a varios ingenieros ubicados en una sala cuadrada. El objetivo principal es encontrar automáticamente un programa que controle los movimientos del robot y le permita obtener la mayor cantidad posible de puntos entregando galletas a los ingenieros. En lugar de programar manualmente una ruta o estrategia específica, la Programación Genética genera y mejora diferentes programas mediante un proceso evolutivo inspirado en la selección natural.

```
Python
import random
import copy
import matplotlib.pyplot as plt

# CONFIGURACION (paso 5: parámetros de control de la corrida)

TAMANO_SALA = 8          # sala cuadrada de TAMANO_SALA x TAMANO_SALA
MAX_PASOS = 60           # pasos disponibles por corrida del robot
TAM_POBLACION = 80
NUM_GENERACIONES = 60
PROB_CRUCE = 0.8
PROB_MUTACION = 0.2
TAM_TORNEO = 3
NUM_ELITE = 2            # elitismo: mejores individuos que pasan directo
PROFUNDIDAD_INICIAL = 4
PROFUNDIDAD_MAXIMA = 7   # límite para evitar crecimiento excesivo (bloat)
PENALIZACION_TAMANO = 0.02 # castiga árboles innecesariamente grandes

# Ingenieros: posición (x, y) y puntos que otorgan al ser atendidos
INGENIEROS_INICIALES = [
    {"x": 2, "y": 2, "puntos": 10},
    {"x": 5, "y": 1, "puntos": 10},
    {"x": 6, "y": 6, "puntos": 10},
    {"x": 1, "y": 5, "puntos": 10},
    {"x": 4, "y": 7, "puntos": 10},
    {"x": 7, "y": 3, "puntos": 10},
]
```

```

# Direcciones: 0=Norte, 1=Este, 2=Sur, 3=Oeste (sentido horario)
DIRECCIONES = [(0, -1), (1, 0), (0, 1), (-1, 0)]
NOMBRE_DIRECCION = ["N", "E", "S", "O"]

# MODELO DEL MUNDO: SALA CUADRADA Y ROBOT

class Sala:
    def __init__(self, tamano, ingenieros):
        self.tamano = tamano
        # copia profunda para no modificar la lista original entre corridas
        self.ingenieros = [dict(i, servido=False) for i in ingenieros]
        self.puntos_obtenidos = 0

    def ingeniero_en(self, x, y):
        for ing in self.ingenieros:
            if not ing["servido"] and ing["x"] == x and ing["y"] == y:
                return ing
        return None

class Robot:
    def __init__(self, x=0, y=0, direccion=0):
        self.x = x
        self.y = y
        self.direccion = direccion # indice en DIRECCIONES
        self.camino = [(x, y)] # para poder visualizar el recorrido

# PASO 2: CONJUNTO DE TERMINALES (acciones atómicas, aridad 0)

TERMINALES = ["avanza", "gira_izquierda", "gira_derecha", "entrega_galleta"]

def avanza(robot, sala, contador):
    if contador[0] <= 0:
        return
    contador[0] -= 1
    dx, dy = DIRECCIONES[robot.direccion]
    nx, ny = robot.x + dx, robot.y + dy
    if 0 <= nx < sala.tamano and 0 <= ny < sala.tamano:
        robot.x, robot.y = nx, ny
        robot.camino.append((robot.x, robot.y))

def gira_izquierda(robot, sala, contador):
    if contador[0] <= 0:

```

```

        return
    contador[0] -= 1
    robot.direccion = (robot.direccion - 1) % 4

def gira_derecha(robot, sala, contador):
    if contador[0] <= 0:
        return
    contador[0] -= 1
    robot.direccion = (robot.direccion + 1) % 4

def entrega_galleta(robot, sala, contador):
    if contador[0] <= 0:
        return
    contador[0] -= 1
    ing = sala.ingeniero_en(robot.x, robot.y)
    if ing is not None:
        ing["servido"] = True
        sala.puntos_obtenidos += ing["puntos"]

ACCIONES_TERMINALES = {
    "avanza": avanza,
    "gira_izquierda": gira_izquierda,
    "gira_derecha": gira_derecha,
    "entrega_galleta": entrega_galleta,
}

# PASO 3: CONJUNTO DE FUNCIONES PRIMITIVAS (aridad > 0)

FUNCIONES = {
    "si_ingeniero_adelante": 2,
    "secuencia2": 2,
    "secuencia3": 3,
}

def ingeniero_adelante(robot, sala):
    dx, dy = DIRECCIONES[robot.direccion]
    nx, ny = robot.x + dx, robot.y + dy
    if not (0 <= nx < sala.tamano and 0 <= ny < sala.tamano):
        return False
    return sala.ingeniero_en(nx, ny) is not None

```

```

# REPRESENTACION DEL ARBOL (cromosoma): (nombre, [hijos])

def generar_arbol(prof_max, metodo="grow", prof_actual=0):
    """Genera un árbol aleatorio (equivalente a Genera_Subárbol del
    documento, aplicado recursivamente)."""
    en_hoja = prof_actual >= prof_max or (
        metodo == "grow" and prof_actual > 0 and random.random() < 0.35
    )
    if en_hoja:
        return (random.choice(TERMINALES), [])
    nombre = random.choice(list(FUNCIONES.keys()))
    aridad = FUNCIONES[nombre]
    hijos = [generar_arbol(prof_max, metodo, prof_actual + 1) for _ in range(aridad)]
    return (nombre, hijos)

def generar_poblacion_inicial(tam_poblacion, prof_max):
    """Ramped half-and-half simplificado: mitad con método 'full',
    mitad con método 'grow', variando la profundidad máxima."""
    poblacion = []
    for i in range(tam_poblacion):
        metodo = "full" if i % 2 == 0 else "grow"
        profundidad = random.randint(2, prof_max)
        poblacion.append(generar_arbol(profundidad, metodo))
    return poblacion

def ejecutar(nodo, robot, sala, contador):
    """Ejecuta (interpreta) el árbol-programa sobre el robot."""
    if contador[0] <= 0:
        return
    nombre, hijos = nodo
    if nombre in ACCIONES_TERMINALES:
        ACCIONES_TERMINALES[nombre](robot, sala, contador)
    elif nombre == "secuencia2":
        ejecutar(hijos[0], robot, sala, contador)
        ejecutar(hijos[1], robot, sala, contador)
    elif nombre == "secuencia3":
        for h in hijos:
            ejecutar(h, robot, sala, contador)
    elif nombre == "si_ingeniero_adelante":
        if ingeniero_adelante(robot, sala):
            ejecutar(hijos[0], robot, sala, contador)
    else:

```

```

        ejecutar(hijos[1], robot, sala, contador)
    else:
        raise ValueError(f" Nodo desconocido: {nombre}")

def contar_nodos(nodo):
    nombre, hijos = nodo
    return 1 + sum(contar_nodos(h) for h in hijos)

def profundidad(nodo):
    nombre, hijos = nodo
    if not hijos:
        return 1
    return 1 + max(profundidad(h) for h in hijos)

def arbol_a_texto(nodo):
    nombre, hijos = nodo
    if not hijos:
        return nombre
    return f"{nombre}(" + ", ".join(arbol_a_texto(h) for h in hijos) + ")"

# PASO 4: FUNCION DE APTITUD

def evaluar_aptitud(arbol, tamano_sala=TAMANO_SALA,
                    ingenieros_iniciales=INGENIEROS_INICIALES,
                    max_pasos=MAX_PASOS, registrar_camino=False):
    """Corre el robot controlado por 'arbol' dentro de la sala y devuelve
    los puntos obtenidos, penalizados levemente por el tamaño del árbol."""
    sala = Sala(tamano_sala, ingenieros_iniciales)
    robot = Robot(0, 0, 0)
    contador = [max_pasos]

    # El árbol se ejecuta repetidamente (como un bucle de control) hasta
    # agotar los pasos disponibles, igual que en el problema clásico del
    # "hormiga de Santa Fe" en PG.
    while contador[0] > 0:
        antes = contador[0]
        ejecutar(arbol, robot, sala, contador)
        if contador[0] == antes:
            # el árbol no consumió ningún paso (caso degenerado) -> evitar bucle infinito
            break

    aptitud = sala.puntos_obtenidos - PENALIZACION_TAMANO * contar_nodos(arbol)

```

```

    if registrar_camino:
        return aptitud, sala, robot
    return aptitud

# OPERADORES GENETICOS: SELECCION, CRUCE Y MUTACION

def seleccion_torneo(poblacion, aptitudes, k=TAM_TORNEO):
    participantes = random.sample(range(len(poblacion)), k)
    mejor = max(participantes, key=lambda i: aptitudes[i])
    return poblacion[mejor]

def obtener_camino(nodo, camino_actual=()):
    """Devuelve la lista de 'caminos' (rutas de índices) hacia cada nodo
    del árbol, recorriéndolo en preorden (igual que en la sección 4.7 del
    documento, donde se numeran los nodos en preorden para el cruce)."""
    caminos = [camino_actual]
    _, hijos = nodo
    for i, h in enumerate(hijos):
        caminos.extend(obtener_camino(h, camino_actual + (i,)))
    return caminos

def obtener_subarbol(arbol, camino):
    nodo = arbol
    for i in camino:
        nodo = nodo[1][i]
    return nodo

def reemplazar_subarbol(arbol, camino, nuevo_subarbol):
    if not camino:
        return copy.deepcopy(nuevo_subarbol)
    nombre, hijos = arbol
    hijos = list(hijos)
    idx = camino[0]
    hijos[idx] = reemplazar_subarbol(hijos[idx], camino[1:], nuevo_subarbol)
    return (nombre, hijos)

def cruce(padre1, padre2, prof_maxima=PROFUNDIDAD_MAXIMA):
    """Cruce de árboles (sección 4.7): se elige un punto de cruce
    aleatorio en cada padre y se intercambian los subárboles."""

```

```

    p1 = copy.deepcopy(padre1)
    p2 = copy.deepcopy(padre2)
    camino1 = random.choice(obtener_caminos(p1))
    camino2 = random.choice(obtener_caminos(p2))
    sub1 = obtener_subarbol(p1, camino1)
    sub2 = obtener_subarbol(p2, camino2)
    hijo1 = reemplazar_subarbol(p1, camino1, sub2)
    hijo2 = reemplazar_subarbol(p2, camino2, sub1)
    # control de crecimiento excesivo de profundidad (ver comentario del
    # documento tras la figura 4.4)
    if profundidad(hijo1) > prof_maxima:
        hijo1 = p1
    if profundidad(hijo2) > prof_maxima:
        hijo2 = p2
    return hijo1, hijo2

def mutacion(arbol, prof_maxima=PROFUNDIDAD_MAXIMA, prof_subarbol_nuevo=3):
    """Reemplaza un subárbol elegido al azar por uno nuevo generado
    aleatoriamente (equivalente al cambio de gen descrito en el
    documento)."""
    nuevo = copy.deepcopy(arbol)
    camino = random.choice(obtener_caminos(nuevo))
    subarbol_nuevo = generar_arbol(prof_subarbol_nuevo, "grow")
    resultado = reemplazar_subarbol(nuevo, camino, subarbol_nuevo)
    if profundidad(resultado) > prof_maxima:
        return arbol
    return resultado

# ALGORITMO PRINCIPAL DE PG (sección 4.3: pasos 1-3 del algoritmo general)

def algoritmo_pg():
    poblacion = generar_poblacion_inicial(TAM_POBLACION, PROFUNDIDAD_INICIAL)
    historial_mejor = []
    historial_promedio = []
    mejor_individuo_global = None
    mejor_apertura_global = float("-inf")

    for gen in range(NUM_GENERACIONES):
        aptitudes = [evaluar_apertura(ind) for ind in poblacion]

        idx_ordenados = sorted(range(len(poblacion)), key=lambda i: aptitudes[i],
reverse=True)
        mejor_gen = aptitudes[idx_ordenados[0]]

```

```

    promedio_gen = sum(aptitudes) / len(aptitudes)
    historial_mejor.append(mejor_gen)
    historial_promedio.append(promedio_gen)

    if mejor_gen > mejor_aptitud_global:
        mejor_aptitud_global = mejor_gen
        mejor_individuo_global = copy.deepcopy(poblacion[idx_ordenados[0]])

    if gen % 5 == 0 or gen == NUM_GENERACIONES - 1:
        print(f"Generación {gen:3d} | mejor aptitud = {mejor_gen:6.2f} | promedio = {promedio_gen:6.2f}")

    # Elitismo: los mejores pasan directo a la siguiente generación
    nueva_poblacion = [copy.deepcopy(poblacion[i]) for i in
idx_ordenados[:NUM_ELITE]]

    # Reproducción, cruce y mutación hasta completar la población
    while len(nueva_poblacion) < TAM_POBLACION:
        padre1 = seleccion_torneo(poblacion, aptitudes)
        padre2 = seleccion_torneo(poblacion, aptitudes)

        if random.random() < PROB_CRUCE:
            hijo1, hijo2 = cruce(padre1, padre2)
        else:
            hijo1, hijo2 = copy.deepcopy(padre1), copy.deepcopy(padre2)

        if random.random() < PROB_MUTACION:
            hijo1 = mutacion(hijo1)
        if random.random() < PROB_MUTACION:
            hijo2 = mutacion(hijo2)

        nueva_poblacion.append(hijo1)
        if len(nueva_poblacion) < TAM_POBLACION:
            nueva_poblacion.append(hijo2)

    poblacion = nueva_poblacion

    return mejor_individuo_global, mejor_aptitud_global, historial_mejor,
historial_promedio

# VISUALIZACION DEL RESULTADO

def dibujar_sala(sala, robot, tamano):
    print("\nRecorrido final del robot (S = inicio, R = robot, I = ingeniero servido, i = sin servir):")

```

```

for y in range(tamano):
    fila = ""
    for x in range(tamano):
        ing = None
        for i in sala.ingenieros:
            if i["x"] == x and i["y"] == y:
                ing = i
        if x == robot.x and y == robot.y:
            fila += " R "
        elif ing is not None:
            fila += " I " if ing["servido"] else " i "
        elif (x, y) == (0, 0):
            fila += " S "
        else:
            fila += " . "
    print(fila)

def graficar_evolucion(historial_mejor, historial_promedio, ruta_salida):
    plt.figure(figsize=(7, 4.5))
    plt.plot(historial_mejor, label="Mejor aptitud", linewidth=2)
    plt.plot(historial_promedio, label="Aptitud promedio", linestyle="--")
    plt.xlabel("Generación")
    plt.ylabel("Aptitud (puntos)")
    plt.title("Evolución de la aptitud - Robot repartidor de galletas (PG)")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig(ruta_salida, dpi=150)
    print(f"\nGráfica de evolución guardada en: {ruta_salida}")

if __name__ == "__main__":
    random.seed(42) # reproducibilidad

    print("Sala:", TAMANO_SALA, "x", TAMANO_SALA)
    print("Ingenieros:", [(i["x"], i["y"]) for i in INGENIEROS_INICIALES])
    print("Iniciando evolución...\n")

    mejor_arbol, mejor_aptitud, hist_mejor, hist_prom = algoritmo_pg()

    print("\n=== RESULTADO FINAL ===")
    print("Mejor aptitud encontrada :", round(mejor_aptitud, 2))
    print("Tamaño del árbol          :", contar_nodos(mejor_arbol), "nodos")
    print("Profundidad del árbol       :", profundidad(mejor_arbol))
    print("Programa (preorden)         :", arbol_a_texto(mejor_arbol))

```

```

    aptitud_final, sala_final, robot_final = evaluar_aptitud(
        mejor_arbol, registrar_camino=True
    )
    ingenieros_servidos = sum(1 for i in sala_final.ingenieros if i["servido"])
    print(f"Ingenieros atendidos      : {ingenieros_servidos} de
{len(sala_final.ingenieros)}")
    print(f"Puntos obtenidos          : {sala_final.puntos_obtenidos}")

    dibujar_sala(sala_final, robot_final, TAMANO_SALA)
    graficar_evolucion(hist_mejor, hist_prom, "evolucion_aptitud.png")

```

Explicación del código

Inicialmente se define el entorno del problema. La sala tiene un tamaño de 8×8 celdas y contiene seis ingenieros ubicados en posiciones específicas. Cada ingeniero otorga 10 puntos cuando recibe una galleta. Además, se establece un número máximo de pasos que el robot puede ejecutar durante cada simulación. Estos parámetros representan las condiciones bajo las cuales serán evaluados todos los programas generados por el algoritmo.

Para representar el entorno se utilizan dos clases principales: Sala y Robot. La clase Sala almacena el tamaño del escenario, la posición de los ingenieros y el estado de cada uno de ellos, indicando si ya recibió una galleta o no. También registra la cantidad total de puntos obtenidos durante la simulación. Por su parte, la clase Robot almacena la posición actual del robot, la dirección hacia la cual está orientado y el recorrido realizado. Gracias a estas dos clases es posible simular completamente el comportamiento del sistema.

Siguiendo la metodología de Programación Genética, el siguiente paso consiste en definir el conjunto de terminales. Los terminales representan las acciones básicas que el robot puede ejecutar. En este caso existen cuatro acciones: avanzar una celda, girar a la izquierda, girar a la derecha y entregar una galleta. Estas acciones son las unidades mínimas de comportamiento que posteriormente serán combinadas para formar programas más complejos. Cada vez que una de estas acciones es ejecutada se consume uno de los pasos disponibles para la simulación.

Posteriormente se define el conjunto de funciones primitivas. Estas funciones permiten construir estructuras más elaboradas a partir de los terminales. La función `secuencia2` ejecuta dos acciones consecutivas, mientras que `secuencia3` ejecuta tres acciones en orden. Adicionalmente, la función `si_ingeniero_adelante` introduce una capacidad de decisión, ya que permite verificar si existe un ingeniero justo delante del robot y ejecutar una acción diferente dependiendo del resultado. Estas funciones son fundamentales porque permiten que los programas evolucionados desarrollen comportamientos adaptativos en lugar de limitarse a realizar movimientos aleatorios.

Cada individuo de la población es representado mediante un árbol. En Programación Genética los árboles cumplen el papel que desempeñan los cromosomas en los algoritmos genéticos tradicionales. Los nodos internos del árbol corresponden a funciones primitivas y las hojas corresponden a terminales. De esta manera, cada árbol representa un programa completo capaz de controlar el comportamiento del robot. Dependiendo de cómo estén organizados los nodos y las ramas, el robot ejecutará estrategias diferentes para desplazarse y entregar galletas.

La población inicial se genera aleatoriamente utilizando una estrategia conocida como Ramped Half-and-Half. Este método combina árboles generados mediante las técnicas Full y Grow. Algunos individuos tienen estructuras completamente desarrolladas y otros presentan formas más irregulares. Esta combinación favorece la diversidad genética desde el comienzo de la evolución, lo cual aumenta las probabilidades de encontrar soluciones de buena calidad.

Una vez generados los árboles, estos deben ser ejecutados para evaluar su desempeño. La función ejecutar actúa como un intérprete que recorre el árbol y realiza las acciones correspondientes. Si encuentra un terminal, ejecuta directamente la acción asociada. Si encuentra una secuencia, ejecuta sus hijos en orden. Si encuentra una condición, evalúa la situación del entorno y decide cuál rama ejecutar. Gracias a este mecanismo, cada árbol se comporta como un programa autónomo capaz de controlar al robot dentro de la sala.

La calidad de cada individuo se mide mediante la función de aptitud. Para calcularla se crea una simulación completa donde el robot intenta repartir galletas utilizando el programa representado por el árbol. El árbol se ejecuta repetidamente hasta que se consumen todos los pasos disponibles. La aptitud corresponde a los puntos obtenidos por las entregas realizadas menos una pequeña penalización proporcional al tamaño del árbol. Esta penalización tiene como objetivo evitar que los programas crezcan excesivamente sin aportar mejoras reales en el desempeño, fenómeno conocido como bloat.

Después de evaluar a todos los individuos, el algoritmo aplica un mecanismo de selección por torneo. En este proceso se eligen varios individuos al azar y se selecciona como ganador aquel que tenga la mejor aptitud. Los individuos seleccionados se convierten en padres de la siguiente generación. Este método favorece a las mejores soluciones, pero al mismo tiempo mantiene cierto nivel de diversidad dentro de la población.

La generación de nuevos individuos se realiza mediante los operadores genéticos de cruce y mutación. El cruce consiste en seleccionar aleatoriamente un subárbol de cada padre e intercambiarlos. De esta manera, los descendientes heredan características de ambos progenitores. Posteriormente puede aplicarse mutación, que consiste en reemplazar un subárbol por otro generado aleatoriamente. Estos dos operadores permiten explorar nuevas soluciones y combinar estrategias exitosas encontradas durante la evolución. Además, el código controla la profundidad máxima de los árboles para evitar que crezcan indefinidamente.

El proceso evolutivo completo se desarrolla durante un número fijo de generaciones. En cada generación se evalúa toda la población, se identifican los mejores individuos y se conserva una pequeña cantidad de ellos mediante elitismo. Posteriormente se generan nuevos individuos utilizando selección, cruce y mutación hasta completar nuevamente el tamaño de la población. A medida que avanzan las generaciones, las soluciones tienden a mejorar porque los programas con mayor aptitud tienen más probabilidades de transmitir sus características a los descendientes.

Finalmente, cuando termina el número establecido de generaciones, el algoritmo devuelve el mejor árbol encontrado durante toda la ejecución. Además de mostrar la aptitud obtenida, también informa el tamaño y la profundidad del árbol, la cantidad de ingenieros atendidos y los puntos logrados. Adicionalmente se genera una representación visual del estado final de la sala y una gráfica que muestra cómo evolucionó la aptitud de la población a lo largo de las generaciones. Esto permite analizar el desempeño del algoritmo y verificar si efectivamente logró aprender una estrategia eficiente para repartir galletas.

En conclusión, este código implementa de manera completa un sistema de Programación Genética en el que cada individuo representa un programa de control para un robot. Mediante un proceso de evolución basado en selección, cruce y mutación, el algoritmo busca automáticamente estrategias cada vez más eficientes para recorrer la sala y entregar galletas a los ingenieros. El resultado final es un programa capaz de maximizar la cantidad de puntos obtenidos sin que el programador tenga que definir explícitamente la ruta o las decisiones que debe tomar el robot.

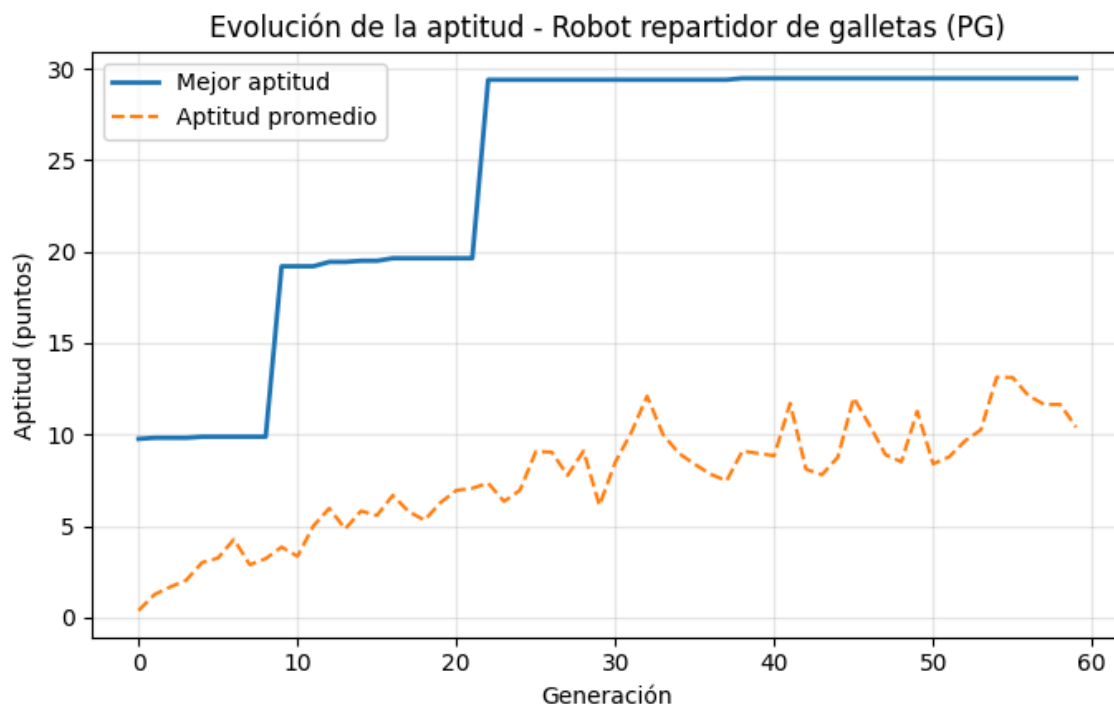
Resultados obtenidos

```
Sala: 8 x 8
Ingenieros: [(2, 2), (5, 1), (6, 6), (1, 5), (4, 7), (7, 3)]
Iniciando evolución...

Generación 0 | mejor aptitud = 9.76 | promedio = 0.37
Generación 5 | mejor aptitud = 9.88 | promedio = 3.24
Generación 10 | mejor aptitud = 19.20 | promedio = 3.34
Generación 15 | mejor aptitud = 19.50 | promedio = 5.58
Generación 20 | mejor aptitud = 19.64 | promedio = 6.94
Generación 25 | mejor aptitud = 29.40 | promedio = 9.07
Generación 30 | mejor aptitud = 29.40 | promedio = 8.49
Generación 35 | mejor aptitud = 29.40 | promedio = 8.37
Generación 40 | mejor aptitud = 29.48 | promedio = 8.85
Generación 45 | mejor aptitud = 29.48 | promedio = 12.02
Generación 50 | mejor aptitud = 29.48 | promedio = 8.39
Generación 55 | mejor aptitud = 29.48 | promedio = 13.12
Generación 59 | mejor aptitud = 29.48 | promedio = 10.40
```

En esta ejecución los resultados son muy similares a la corrida anterior. El algoritmo comenzó con una mejor aptitud de 9.76 puntos y terminó alcanzando una aptitud máxima de 29.48 puntos. Esto indica que, a medida que avanzaron las generaciones, la Programación Genética fue encontrando programas cada vez más eficientes para controlar el movimiento del robot y mejorar la entrega de galletas a los ingenieros. La aptitud promedio de la

población también aumentó progresivamente, pasando de 0.37 en la generación inicial a valores cercanos a 10 puntos al final de la ejecución, lo que demuestra una mejora general de los individuos de la población.



La gráfica de evolución muestra tres etapas claramente diferenciadas. Durante las primeras generaciones la mejor aptitud se mantuvo cercana a los 10 puntos, lo que corresponde a individuos capaces de atender únicamente un ingeniero. Posteriormente, alrededor de la generación 10, se observa un incremento hasta aproximadamente 19.5 puntos, indicando que comenzaron a aparecer programas capaces de atender dos ingenieros. Finalmente, cerca de la generación 23 se produce el mayor salto evolutivo, alcanzando una aptitud de 29.48 puntos, valor que se mantiene estable hasta el final de la corrida. Esta estabilidad indica que el algoritmo convergió hacia la mejor solución encontrada y que no logró descubrir una estrategia superior en las generaciones restantes.

```
=== RESULTADO FINAL ===  
Mejor aptitud encontrada : 29.48  
Tamaño del árbol       : 26 nodos  
Profundidad del árbol  : 7  
Ingenieros atendidos   : 3 de 6  
Puntos obtenidos       : 30
```

El mejor individuo obtenido tiene un tamaño de 26 nodos y una profundidad de 7 niveles, alcanzando el límite máximo de profundidad permitido por la configuración del algoritmo. Durante la simulación este programa logró atender a 3 de los 6 ingenieros presentes en la sala, obteniendo 30 puntos reales. Sin embargo, debido a la penalización aplicada al tamaño del árbol para evitar soluciones excesivamente grandes, la aptitud final reportada fue de 29.48 puntos.

Recorrido final del robot (S = inicio, R = robot, I = ingeniero servido, i = sin servir):

```
S . . . . . . . .  
. . . . . I . .  
. . I . . . . .  
. . . . . . . I  
. . . . . . R .  
. i . . . . . .  
. . . . . i .  
. . . . i . . .
```

Al analizar el mapa final de la sala se observa que tres ingenieros aparecen marcados como atendidos, mientras que los otros tres permanecen sin recibir galletas. Esto indica que la estrategia evolucionada logró recorrer de manera efectiva una parte del entorno, pero no consiguió explorar toda la sala ni alcanzar a todos los ingenieros dentro de los 60 pasos disponibles. El robot finaliza su recorrido aproximadamente en la zona central de la sala, lo que sugiere que la solución encontrada concentra sus movimientos en una región específica del escenario.

En términos generales, los resultados demuestran que la Programación Genética fue capaz de aprender automáticamente una estrategia funcional para el problema planteado. La mejora constante de la aptitud máxima y promedio evidencia que el proceso evolutivo permitió generar programas cada vez más efectivos, logrando finalmente atender la mitad de los ingenieros disponibles y obtener el 50 % de la puntuación máxima posible bajo las condiciones establecidas.

3. Detección de fraudes. Genere aleatoriamente un archivo con los siguientes datos: monto en pesos, hora del día, distancia a su residencia, ingreso mensual; y fraude sí o no. Revíselo y haga cambios para que en su concepto sea más real. Encuentre un modelo que determine un posible fraude.

Para resolver el ejercicio de detección de fraudes mediante Programación Genética (PG), se desarrolló un sistema capaz de generar datos sintéticos de transacciones bancarias y posteriormente evolucionar un modelo que permita clasificar si una transacción corresponde a un fraude o no.

La solución sigue los principios de Programación Genética descritos en el documento del curso: los individuos de la población son programas representados como árboles, se evalúan mediante una función de aptitud, y evolucionan a través de operadores genéticos como cruce y mutación hasta encontrar una expresión matemática capaz de diferenciar transacciones legítimas de transacciones fraudulentas.

Las variables utilizadas para cada transacción son:

- Monto de la transacción.
- Hora del día.
- Distancia respecto a la residencia.
- Ingreso mensual.

- Etiqueta de fraude (Sí o No).

El objetivo es encontrar automáticamente una función matemática que permita predecir si una nueva transacción es sospechosa.

Python

```
import matplotlib.pyplot as plt
from IPython.display import display
import copy
import math
import random
from pathlib import Path

import numpy as np
import pandas as pd

COLUMNAS = ["monto_pesos", "hora_dia", "distancia_residencia_km", "ingreso_mensual"]

def generar_datos(n=3000, tasa_fraude=0.08, semilla=42, ruta="transacciones.csv"):
    """
    Genera transacciones sinteticas con un comportamiento 'realista':

    - Ingreso mensual: lognormal, centrado en ~3 millones COP (minimo 1.4 M).
    - Transaccion legitima: monto es una fraccion pequena del ingreso, ocurre
      sobre todo de dia, y cerca de la residencia.
    - Fraude: monto mayor respecto al ingreso, mas frecuente de madrugada y
      lejos de la residencia.
    - Ruido (para que no sea trivial):
      * 12 % de los fraudes 'se parecen' a una transaccion normal
      * 1.5 % de las legitimas tienen rasgos sospechosos (viaje, compra grande)
    - Desbalance: ~8 % de fraudes (en la vida real suele ser incluso menor).
    """
    rng = np.random.default_rng(semilla)
    es_fraude = rng.random(n) < tasa_fraude

    ingreso = np.clip(rng.lognormal(mean=math.log(3_000_000), sigma=0.55, size=n),
                      1_400_000, 25_000_000)

    monto = np.empty(n)
    hora = np.empty(n)
    dist = np.empty(n)

    # --- legitimas
    L = ~es_fraude
    nL = L.sum()
    monto[L] = ingreso[L] * rng.lognormal(math.log(0.03), 0.9, nL)
    # horas: mezcla centrada a medio dia y en la tarde-noche
```

```

hora[L] = np.where(rng.random(nL) < 0.6,
                  rng.normal(13, 3.5, nL), rng.normal(19, 2.5, nL)) % 24
dist[L] = rng.lognormal(math.log(4), 0.9, nL)

# --- fraudes
F = es_fraude
nF = F.sum()
monto[F] = ingreso[F] * rng.lognormal(math.log(0.30), 0.8, nF)
hora[F] = np.where(rng.random(nF) < 0.7,
                  rng.normal(2.5, 2.0, nF), rng.uniform(0, 24, nF)) % 24
dist[F] = rng.lognormal(math.log(60), 0.9, nF)

# --- ruido: fraudes que parecen normales
idxF = np.where(F)[0]
camuflados = rng.choice(idxF, size=int(0.12 * len(idxF)), replace=False)
monto[camuflados] = ingreso[camuflados] * rng.lognormal(math.log(0.03), 0.6,
len(camuflados))
hora[camuflados] = rng.normal(14, 3, len(camuflados)) % 24
dist[camuflados] = rng.lognormal(math.log(5), 0.6, len(camuflados))

# --- ruido: legitimas con rasgos sospechosos (viaje, compra grande)
idxL = np.where(L)[0]
raras = rng.choice(idxL, size=int(0.015 * len(idxL)), replace=False)
monto[raras] = ingreso[raras] * rng.lognormal(math.log(0.25), 0.5, len(raras))
dist[raras] = rng.lognormal(math.log(40), 0.6, len(raras))

df = pd.DataFrame({
    "monto_pesos": np.round(monto, -2).astype(int), # redondeo a centenas de pesos
    "hora_dia": np.round(hora, 2),
    "distancia_residencia_km": np.round(dist, 1),
    "ingreso_mensual": np.round(ingreso, -3).astype(int),
    "fraude": np.where(es_fraude, "si", "no"),
})
df.to_csv(ruta, index=False)
return df

def _div(a, b):
    """Division protegida: mantiene la propiedad de CLAUSURA."""
    return np.where(np.abs(b) > 1e-6, a / np.where(np.abs(b) > 1e-6, b, 1.0), 1.0)

FUNCIONES = {
    # nombre: (aridad, funcion, simbolo infijo o None)
    "add": (2, np.add, "+"),
    "sub": (2, np.subtract, "-"),
    "mul": (2, np.multiply, "*"),
    "div": (2, _div, "/"),

```

```

    "max": (2, np.maximum, None),
    "min": (2, np.minimum, None),
    "sin": (1, np.sin, None),
    "cos": (1, np.cos, None),
}

NOMBRES_FUNC = list(FUNCIONES)
TERMINALES = ["x0", "x1", "x2", "x3"] # monto, hora, distancia, ingreso
ALIAS = {"x0": "monto", "x1": "hora", "x2": "distancia", "x3": "ingreso"}

def preparar_variables(df, escalas=None):
    """
    Escala las variables para que la PG trabaje con numeros de magnitud similar:
    x0 = monto / media_monto
    x1 = hora * 2*pi / 24 (angulo: asi sin/cos pueden representar 'madrugada')
    x2 = distancia / media_distancia
    x3 = ingreso / media_ingreso
    Devuelve matriz X (n,4) y las escalas usadas (para reutilizarlas en
    validacion/prueba).
    """
    if escalas is None:
        escalas = {
            "monto": df["monto_pesos"].mean(),
            "dist": df["distancia_residencia_km"].mean(),
            "ingreso": df["ingreso_mensual"].mean(),
        }
    X = np.column_stack([
        df["monto_pesos"] / escalas["monto"],
        df["hora_dia"] * 2 * np.pi / 24,
        df["distancia_residencia_km"] / escalas["dist"],
        df["ingreso_mensual"] / escalas["ingreso"],
    ])
    return X, escalas

def terminal_aleatorio(rng):
    if rng.random() < 0.7:
        return rng.choice(TERMINALES)
    return round(rng.uniform(-2, 2), 3) # constante efimera

def generar_arbol(rng, prof_max, metodo="grow"):
    """Metodos 'full' y 'grow' (se combinan: ramped half-and-half)."""
    if prof_max == 0 or (metodo == "grow" and rng.random() < 0.3):
        return terminal_aleatorio(rng)
    f = rng.choice(NOMBRES_FUNC)
    aridad = FUNCIONES[f][0]

```

```

    return [f] + [generar_arbol(rng, prof_max - 1, metodo) for _ in range(aridad)]

def cuenta_nodos(t):
    """Cuenta_Nodos del documento."""
    if not isinstance(t, list):
        return 1
    return 1 + sum(cuenta_nodos(h) for h in t[1:])

def profundidad(t):
    if not isinstance(t, list):
        return 1
    return 1 + max(profundidad(h) for h in t[1:])

def obtener_subarbol(t, idx):
    """Devuelve el subarbol en la posicion idx (preorden, 0 = raiz)."""
    if idx == 0:
        return t
    idx -= 1
    for h in t[1:]:
        n = cuenta_nodos(h)
        if idx < n:
            return obtener_subarbol(h, idx)
        idx -= n
    raise IndexError

def reemplazar_subarbol(t, idx, nuevo):
    """Devuelve una COPIA de t con el nodo idx reemplazado por 'nuevo'."""
    if idx == 0:
        return copy.deepcopy(nuevo)
    resultado = [t[0]]
    idx -= 1
    for h in t[1:]:
        n = cuenta_nodos(h)
        if 0 <= idx < n:
            resultado.append(reemplazar_subarbol(h, idx, nuevo))
        else:
            resultado.append(h)
        idx -= n
    return resultado

def evaluar(t, X):

```

```

"""Ejecuta el arbol-programa sobre todas las filas de X (vectorizado)."""
if not isinstance(t, list):
    if isinstance(t, str):
        return X[:, int(t[1])]
    return np.full(X.shape[0], float(t))
aridad, fn, _ = FUNCIONES[t[0]]
args = [evaluar(h, X) for h in t[1:]]
with np.errstate(all="ignore"):
    r = fn(*args)
return np.clip(np.nan_to_num(r, nan=0.0, posinf=1e6, neginf=-1e6), -1e6, 1e6)

def a_texto(t):
    """Expresion legible (infija) del arbol."""
    if not isinstance(t, list):
        return ALIAS.get(t, str(t)) if isinstance(t, str) else f"{t:g}"
    f = t[0]
    simbolo = FUNCIONES[f][2]
    if simbolo:
        return f"({a_texto(t[1])} {simbolo} {a_texto(t[2])})"
    return f"{f}(" + ", ".join(a_texto(h) for h in t[1:]) + ")"

PENALIZACION_TAMANO = 0.0005 # presion parsimonia: evita arboles gigantes

def predecir(t, X):
    return evaluar(t, X) > 0.0 # salida > 0 -> fraude

def error_balanceado(t, X, y):
    """
    Promedio de la tasa de falsos negativos y de falsos positivos.
    Con datos desbalanceados (8 % fraude) la exactitud simple engana: un
    modelo que siempre diga 'no fraude' tendria 92 % de acierto.
    """
    pred = predecir(t, X)
    fn = np.mean(~pred[y]) if y.any() else 0.0
    fp = np.mean(pred[~y]) if (~y).any() else 0.0
    return 0.5 * (fn + fp)

def aptitud(t, X, y):
    err = error_balanceado(t, X, y) + PENALIZACION_TAMANO * cuenta_nodos(t)
    return 1.0 / (0.1 + err)

def torneo(pob, fits, k, rng):

```

```

candidatos = rng.sample(range(len(pob)), k)
return pob[max(candidatos, key=lambda i: fits[i])]

def cruce(p1, p2, rng, prof_limite):
    """Intercambio de subarboles en puntos de cruce escogidos al azar (preorden)."""
    c1 = rng.randrange(cuenta_nodos(p1))
    c2 = rng.randrange(cuenta_nodos(p2))
    s1, s2 = obtener_subarbol(p1, c1), obtener_subarbol(p2, c2)
    h1 = reemplazar_subarbol(p1, c1, s2)
    h2 = reemplazar_subarbol(p2, c2, s1)
    # el documento advierte que los arboles crecen en profundidad: se limita
    if profundidad(h1) > prof_limite:
        h1 = copy.deepcopy(p1)
    if profundidad(h2) > prof_limite:
        h2 = copy.deepcopy(p2)
    return h1, h2

def mutacion(t, rng, prof_limite):
    """Mutacion de subarbol: reemplaza un nodo por un subarbol aleatorio pequeno."""
    c = rng.randrange(cuenta_nodos(t))
    nuevo = generar_arbol(rng, 2, "grow")
    h = reemplazar_subarbol(t, c, nuevo)
    return h if profundidad(h) <= prof_limite else copy.deepcopy(t)

def metricas(t, X, y):
    pred = predecir(t, X)
    tp = int(np.sum(pred & y))
    tn = int(np.sum(~pred & ~y))
    fp = int(np.sum(pred & ~y))
    fn = int(np.sum(~pred & y))
    exactitud = (tp + tn) / len(y)
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    return dict(tp=tp, tn=tn, fp=fp, fn=fn, exactitud=exactitud,
                precision=precision, recall=recall, f1=f1)

def imprimir_metricas(nombre, m):
    print(f"\n--- {nombre} ---")
    print(f" Matriz de confusion: VP={m['tp']} FN={m['fn']} FP={m['fp']}
VN={m['tn']}")
    print(f" Exactitud={m['exactitud']:.3f} Precision={m['precision']:.3f} "
          f"Recall={m['recall']:.3f} F1={m['f1']:.3f}")

```

```

def evolucionar(Xtr, ytr, Xva, yva, tam_pob=200, generaciones=100, p_cruce=0.7,
               p_mut=0.1, k_torneo=3, n_elite=2, prof_ini=4, prof_max=8,
               semilla=0, verbose=True):
    rng = random.Random(semilla)

    # Generacion 0: poblacion inicial aleatoria (ramped half-and-half)
    pob = []
    for i in range(tam_pob):
        metodo = "full" if i % 2 == 0 else "grow"
        pob.append(generar_arbol(rng, 2 + i % (prof_ini - 1), metodo))

    historia = []
    salon_fama = []          # mejor individuo de cada generacion (hall of fame)

    for g in range(generaciones + 1):
        fits = [aptitud(t, Xtr, ytr) for t in pob]
        orden = sorted(range(tam_pob), key=lambda i: fits[i], reverse=True)
        mejor = pob[orden[0]]
        historia.append(fits[orden[0]])
        salon_fama.append(copy.deepcopy(mejor))

        if verbose and (g % 10 == 0 or g == generaciones):
            m = metricas(mejor, Xva, yva)
            print(f"Gen {g:3d} | apt. entren. = {fits[orden[0]]:.4f} | "
                  f"nodos = {cuenta_nodos(mejor):3d} | F1 valid. = {m['f1']:.3f}")

        if g == generaciones:
            break

    # Nueva poblacion: elitismo + reproduccion / cruce / mutacion
    nueva = [copy.deepcopy(pob[i]) for i in orden[:n_elite]]
    while len(nueva) < tam_pob:
        r = rng.random()
        if r < p_cruce:
            h1, h2 = cruce(torneo(pob, fits, k_torneo, rng),
                          torneo(pob, fits, k_torneo, rng), rng, prof_max)
            nueva.extend([h1, h2])
        elif r < p_cruce + p_mut:
            nueva.append(mutacion(torneo(pob, fits, k_torneo, rng), rng, prof_max))
        else:
            # reproduccion (copia)
            nueva.append(copy.deepcopy(torneo(pob, fits, k_torneo, rng)))
    pob = nueva[:tam_pob]

    # Resultado: de los mejores de cada generacion, el que mejor generaliza en VALIDACION
    # (desempate: arbol mas pequeno)

```

```

def clave(t):
    return (error_balanceado(t, Xva, yva), cuenta_nodos(t))
mejor_final = min(salon_fama, key=clave)
return mejor_final, historia

# ----- PARAMETROS (editables) -----
N_TRANSACCIONES = 3000
SEMILLA          = 42
POBLACION        = 200
GENERACIONES     = 100
P_CRUCE          = 0.7
P_MUTACION       = 0.1
RUTA_CSV         = "transacciones.csv"
# -----

# --- datos
df = generar_datos(N_TRANSACCIONES, semilla=SEMILLA, ruta=RUTA_CSV)
print(f"Archivo '{RUTA_CSV}' generado con {len(df)} transacciones.")
print(f"Fraudes: {(df['fraude'] == 'si').sum()} ({(df['fraude'] == 'si').mean():.1%})")
display(df.head(10))
display(df.describe().round(1))

# --- particion estratificada 60 / 20 / 20
rng_np = np.random.default_rng(SEMILLA)
y_all = (df["fraude"] == "si").to_numpy()
idx_tr, idx_va, idx_te = [], [], []
for clase in (True, False):
    idx = rng_np.permutation(np.where(y_all == clase)[0])
    a, b = int(0.6 * len(idx)), int(0.8 * len(idx))
    idx_tr += list(idx[:a]); idx_va += list(idx[a:b]); idx_te += list(idx[b:])
tr, va, te = df.iloc[idx_tr], df.iloc[idx_va], df.iloc[idx_te]

Xtr, escalas = preparar_variables(tr)
Xva, _ = preparar_variables(va, escalas)
Xte, _ = preparar_variables(te, escalas)
ytr, yva, yte = (d["fraude"].eq("si").to_numpy() for d in (tr, va, te))

# --- evolucion
print("\n=== Evolucion ===")
mejor, historia = evolucionar(Xtr, ytr, Xva, yva, tam_pob=POBLACION,
                              generaciones=GENERACIONES, p_cruce=P_CRUCE,
                              p_mut=P_MUTACION, semilla=SEMILLA)

# --- resultados
print("\n=== MEJOR MODELO ENCONTRADO ===")
print(f"Regla: salida = ", a_texto(mejor))

```

```

print("          si salida > 0  ->  POSIBLE FRAUDE")
print(f"Nodos: {cuenta_nodos(mejor)} | Profundidad: {profundidad(mejor)}")
print(f"Escalas: monto/{escalas['monto']:.0f}, distancia/{escalas['dist']:.2f}, "
      f"ingreso/{escalas['ingreso']:.0f}, hora*2*pi/24")
imprimir_metricas("Entrenamiento", metricas(mejor, Xtr, ytr))
imprimir_metricas("Validacion", metricas(mejor, Xva, yva))
imprimir_metricas("PRUEBA (datos nunca vistos)", metricas(mejor, Xte, yte))

# --- grafica de la aptitud
plt.figure(figsize=(7, 4))
plt.plot(historia)
plt.xlabel("generaciones"); plt.ylabel("aptitud del mejor (entrenamiento)")
plt.title("Historia de la aptitud - PG deteccion de fraudes")
plt.grid(alpha=.3); plt.tight_layout(); plt.show()

```

Explicación del código

El programa prepara las variables de entrada mediante un proceso de normalización. Esto permite que todas las características trabajen en escalas similares, evitando que variables con valores muy grandes, como el ingreso mensual o el monto de la transacción, dominen el proceso de aprendizaje y afecten negativamente la evolución de las soluciones.

La Programación Genética representa cada posible solución mediante un árbol compuesto por funciones matemáticas y variables de entrada. Las funciones incluyen operaciones como suma, resta, multiplicación, división, máximo, mínimo, seno y coseno, mientras que las terminales corresponden a las variables del problema (monto, hora, distancia e ingreso). Cada árbol generado constituye una regla matemática capaz de clasificar una transacción como fraude o no fraude.

Una vez creada la población inicial de árboles, cada individuo es evaluado mediante una función de aptitud. Esta función mide qué tan bien el árbol clasifica las transacciones del conjunto de entrenamiento. Para ello se calcula el error de clasificación y se transforma en una medida de aptitud, donde los individuos con menor error reciben una mejor valoración. Además, se incluye una penalización para evitar árboles excesivamente grandes y complejos.

El proceso evolutivo se desarrolla durante varias generaciones. En cada iteración se seleccionan los individuos con mejor desempeño mediante selección por torneo. Luego se aplican operadores genéticos de cruce y mutación. El cruce intercambia partes de dos árboles padres para generar nuevas soluciones, mientras que la mutación modifica aleatoriamente una sección del árbol con el fin de introducir diversidad en la población y evitar que el algoritmo quede atrapado en soluciones poco eficientes.

Finalmente, después de completar todas las generaciones, el algoritmo conserva el árbol con la mejor aptitud encontrada. Este árbol representa el modelo de detección de fraudes obtenido

por Programación Genética. Su desempeño se evalúa utilizando métricas como exactitud, precisión, recall y F1-score, permitiendo verificar qué tan efectivo es para identificar transacciones fraudulentas y diferenciar correctamente las operaciones legítimas.

Resultados Obtenidos

	monto_pesos	hora_dia	distancia_residencia_km	ingreso_mensual
count	3000.0	3000.0	3000.0	3000.0
mean	285090.6	14.8	11.9	3540535.3
std	827154.9	5.0	28.8	2085644.6
min	3800.0	0.0	0.1	1400000.0
25%	48500.0	11.7	2.4	2077000.0
50%	104150.0	15.3	4.7	3028500.0
75%	228600.0	18.5	9.4	4306250.0
max	17285100.0	24.0	461.1	20052000.0

Los 3000 registros generados presentan características variadas y realistas. El monto promedio de las transacciones fue de 285.091 pesos, con valores que alcanzan hasta 17,3 millones de pesos, mientras que la distancia promedio a la residencia fue de 11,9 km. Esta variabilidad permite diferenciar comportamientos normales de posibles patrones de fraude.

```

=== Evolucion ===
Gen  0 | apt. entren. = 4.4278 | nodos = 10 | F1 valid. = 0.648
Gen 10 | apt. entren. = 5.5993 | nodos =  8 | F1 valid. = 0.854
Gen 20 | apt. entren. = 5.5993 | nodos =  8 | F1 valid. = 0.854
Gen 30 | apt. entren. = 5.5993 | nodos =  8 | F1 valid. = 0.854
Gen 40 | apt. entren. = 5.5993 | nodos =  8 | F1 valid. = 0.854
Gen 50 | apt. entren. = 5.5993 | nodos =  8 | F1 valid. = 0.854
Gen 60 | apt. entren. = 5.5993 | nodos =  8 | F1 valid. = 0.854
Gen 70 | apt. entren. = 5.5993 | nodos =  8 | F1 valid. = 0.854
Gen 80 | apt. entren. = 5.5993 | nodos =  8 | F1 valid. = 0.854
Gen 90 | apt. entren. = 5.6513 | nodos = 11 | F1 valid. = 0.804
Gen 100 | apt. entren. = 5.6513 | nodos = 11 | F1 valid. = 0.804

```

Durante la evolución, la aptitud del mejor individuo aumentó rápidamente de 4,43 a 5,60 en las primeras generaciones y luego se estabilizó, alcanzando finalmente 5,65. Esto indica que el algoritmo encontró una solución adecuada en pocas generaciones y realizó mejoras menores hasta el final del proceso.

```

=== MEJOR MODELO ENCONTRADO ===
Regla: salida = (monto - (hora / distancia))
      si salida > 0 -> POSIBLE FRAUDE
Nodos: 5 | Profundidad: 3
Escala: monto/299808, distancia/11.65, ingreso/3549723, hora*2*pi/24

--- Entrenamiento ---
Matriz de confusion: VP=116 FN=20 FP=36 VN=1627
Exactitud=0.969 Precision=0.763 Recall=0.853 F1=0.806

--- Validacion ---
Matriz de confusion: VP=39 FN=6 FP=8 VN=547
Exactitud=0.977 Precision=0.830 Recall=0.867 F1=0.848

--- PRUEBA (datos nunca vistos) ---
Matriz de confusion: VP=39 FN=7 FP=8 VN=547
Exactitud=0.975 Precision=0.830 Recall=0.848 F1=0.839

```

La mejor regla encontrada por la Programación Genética fue:

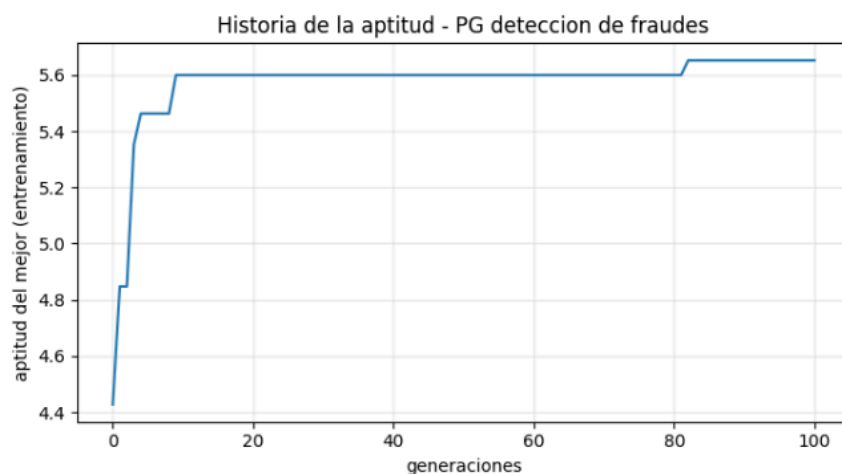
$$\text{salida} = (\text{monto} - (\text{hora} / \text{distancia}))$$

Si la salida es mayor que cero, la transacción se clasifica como posible fraude. Además, el modelo resultante es sencillo, con solo 5 nodos y profundidad 3, lo que facilita su interpretación.

En el conjunto de entrenamiento se obtuvo una exactitud del 96,9 %, un recall del 85,3 % y un F1-score de 0,806, demostrando una alta capacidad para identificar transacciones fraudulentas.

Los resultados de validación fueron similares, con una exactitud del 97,7 % y un F1-score de 0,848, lo que indica que el modelo generaliza adecuadamente y no presenta sobreajuste.

Finalmente, en el conjunto de prueba, compuesto por datos no utilizados durante el entrenamiento, se alcanzó una exactitud del 97,5 %, una precisión del 83,0 %, un recall del 84,8 % y un F1-score de 0,839. Además, el modelo detectó correctamente 39 fraudes y 547 transacciones legítimas, cometiendo únicamente 8 falsos positivos y 7 falsos negativos.



En conclusión, la Programación Genética logró encontrar una regla simple y eficiente, capaz de identificar aproximadamente el 85 % de los fraudes con una exactitud superior al 97 %, mostrando un desempeño sólido y estable en todos los conjuntos de evaluación.